Avaliação das características não funcionais do sistema (SA)

Equipe composta Por

- **Eder J. Tomelin**
- **Yan Kauê Rocha**
- **Vinicius Marcelino**
- **Elias A. Rodrigues**
- **José L. de Souza**
- **Gustavo H. da Silva**
- **Vinicius P. Lessa**

Testes Não Funcionais (Performance, Carga, Estresse)

Testes de Usabilidade e Acessibilidade Digital (UX)

Fundamentos de Segurança da Informação nos Testes

Perguntas Desafio Final

Testes Não Funcionais (Performance, Carga, Estresse)

Porque e como

Os testes não funcionais garantem que um sistema seja rápido, estável e eficiente, mesmo com muitos usuários simultâneos, identificando limitações antes do uso real. Eles avaliam o desempenho em diferentes situações: o teste de performance mede a velocidade de resposta, o de carga verifica o funcionamento dentro do limite esperado e o de estresse testa além desse limite para encontrar falhas, tendo como principal objetivo identificar gargalos que prejudicam o funcionamento do sistema.

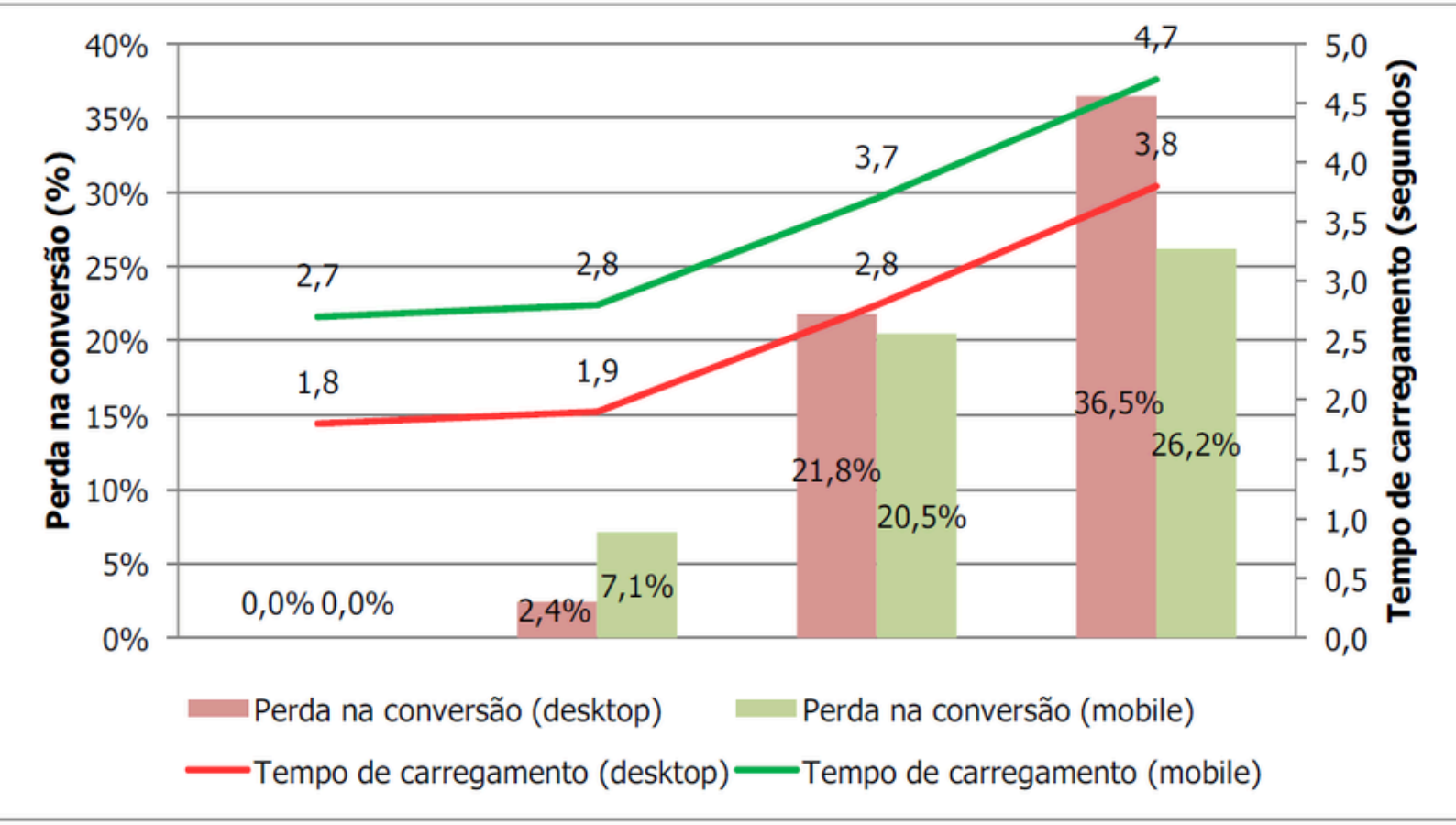
Exemplo de Testes

Teste 1 – Performance
Cenário: Um usuário acessa o site de uma lanchonete e faz um pedido.
Como Testar: Medir o tempo que a página leva para carregar e finalizar o pedido.
Objetivo: Garantir que o sistema responda rapidamente (ex: menos de 2 segundos).

Teste 2 – Carga
Cenário: 1.000 usuários acessando o site ao mesmo tempo.
Como Testar: Simular múltiplos usuários usando ferramentas como JMeter.
Objetivo: Verificar se o sistema suporta o número esperado de acessos sem cair.

Teste 3 – Estresse
Cenário: 10.000 usuários acessando simultaneamente (acima do limite esperado).
Como Testar: Aumentar gradualmente o número de acessos até o sistema falhar.
Objetivo: Descobrir o limite máximo do sistema e identificar o gargalo.

GRÁFICO


Perda na conversão (%)
40%
35%
30%
25%
20%
15%
10%
5%
0%
Tempo de carregamento (segundos)
5,0
4,5
4,0
3,5
3,0
2,5
2,0
1,5
1,0
0,5
0,0
2,7
2,8
3,7
4,7
1,8
1,9
2,8
3,8
0,0% 0,0%
2,4%
7,1%
21,8%
20,5%
36,5%
26,2%
Perda na conversão (desktop)
Perda na conversão (mobile)
Tempo de carregamento (desktop)
Tempo de carregamento (mobile)

Perguntas

1- Qual é a principal diferença entre testes funcionais e não funcionais?

2- O que é teste de performance e quais métricas são usadas?

3- Qual a diferença entre teste de carga e teste de estresse?

4- Qual é o objetivo principal dos testes não funcionais?

5- Cite algumas ferramentas usadas para testes de carga e sua finalidade.

Respostas

1- Testes funcionais verificam o que o sistema faz (ex: se um botão funciona corretamente). Já os testes não funcionais avaliam como o sistema se comporta, como desempenho, tempo de resposta e estabilidade sob carga.

2- Teste de performance mede a velocidade e eficiência do sistema. As principais métricas são: latência (tempo de resposta) e throughput (quantidade de requisições por tempo).

3- Teste de carga avalia o comportamento do sistema sob uso esperado. Já o teste de estresse leva o sistema além do limite para ver como ele se comporta até falhar.

4- O principal objetivo é identificar gargalos de desempenho (bottlenecks), ou seja, descobrir qual parte do sistema está limitando a performance.

5- Exemplos: JMeter (simulação de usuários), Gatling (testes de carga com relatórios detalhados) e K6 (testes automatizados e integração com CI/CD).

Testes de Usabilidade e Acessibilidade Digital (UX)

Por quê:

Um software pode funcionar perfeitamente do ponto de vista técnico e ainda assim falhar completamente com seus usuários. Testes de usabilidade existem para garantir que a interface seja intuitiva, eficiente e agradável — ou seja, que o sistema corresponda ao modelo mental do usuário, não apenas à lógica do desenvolvedor. Sem essa validação, equipes investem tempo construindo funcionalidades que os usuários não conseguem encontrar, formulários que causam confusão e fluxos que levam ao abandono.

Como:

O “como” envolve a aplicação de métodos estruturados de avaliação da interface. Para usabilidade, utilizam-se técnicas como testes com usuários reais e avaliações heurísticas (baseadas nas heurísticas de Jakob Nielsen). Já para acessibilidade, são aplicadas diretrizes como o WCAG (Web Content Accessibility Guidelines), que definem critérios objetivos e testáveis. Também são utilizados recursos como leitores de tela (ex: NVDA, VoiceOver) para validar a experiência de usuários com deficiência.

Gráfico

TESTES DE USABILIDADE E ACESSIBILIDADE DIGITAL
Garantir que o software seja fácil de usar, eficiente, agradável e acessível para todas as pessoas, cumprindo requisitos legais e éticos.
USUÁRIO
USABILIDADE (UX)
Foco na experiência do usuário: facilidade, eficiência e satisfação.
ACESSIBILIDADE
Foco na inclusão: garantir que pessoas com diferentes deficiências consigam usar o sistema.
O QUE AVALIAR
Facilidade e aprender
Eficiência de uso
Satisfação do usuário
Poucos erros e frustrações
O QUE AVALIAR (BASE: WCAG)
Perceptível
Informações devem ser percebidas por todos os sentidos.
Operável
Navegação deve ser possível por teclado e outros meios.
Compreensível
Conteúdo claro, simples e fácil de entender.
Robusto
Compatível co tecnologias assistivas.
MÉTODOS DE TESTE
Teste com usuários reais
Usuários reais executam tarefas enquanto são observados.
Teste heurístico
Especialistas avaliam a interface com base em princípios de usabilidade.
OBJETIVO FINAL
Proporcionar uma experiência digital inclusiva, eficiente e agradável para todos os usuários.
FERRAMENTAS E RECURSOS
Leitores de tela
NVDA (Windows), VoiceOver (macOS/iOS), TalkBack (Android)
Navegação por teclado
Testar usando apenas a tecla TAB e atalhos.
Validadores automáticos
WAVE, axe DevTools Lighthouse, entre outr
MPACTOS POSITIVOS
Inclusão e respeito à diversidade
Conformidade legal (leis e normas)
Melhora da experiência e satisfação
Fortalecimento da imagem e credibilidade

Perguntas desafiadoras

1. O que é usabilidade? Dê um exemplo do dia a dia em que um site ou aplicativo tem boa ou má usabilidade.

2. O que significa a sigla WCAG e qual é o seu objetivo principal?

3. O que é um leitor de tela? Para qual público ele é essencial e cite um exemplo de ferramenta.

4. O que é o atributo alt em uma imagem HTML e por que ele é importante para acessibilidade?

5. Quais são os quatro princípios do WCAG representados pela sigla POUR? Explique cada um em uma frase.

Respostas

1. Usabilidade é a facilidade com que um usuário consegue utilizar uma interface para atingir seus objetivos de forma eficiente e satisfatória. Um exemplo de boa usabilidade é o Google: campo de busca centralizado, único e direto. Um exemplo de má usabilidade é um site de banco com menus confusos, botões pequenos e muitos passos para realizar uma transferência simples.

2. WCAG significa Web Content Accessibility Guidelines (Diretrizes de Acessibilidade para Conteúdo Web). Seu objetivo principal é estabelecer padrões técnicos internacionais para que sites e aplicativos possam ser utilizados por todas as pessoas, incluindo aquelas com deficiências visuais, auditivas, motoras ou cognitivas.

3. **É um software que interpreta o conteúdo exibido na tela e o converte em áudio ou braille, permitindo que o usuário navegue sem precisar enxergar a interface. É essencial para pessoas com deficiência visual. Exemplos de ferramentas: NVDA (Windows, gratuito) e VoiceOver (Mac e iOS, nativo do sistema).**

4. **É um atributo HTML adicionado a imagens que fornece uma descrição textual do conteúdo visual. Exemplo: <img src="cachorro.jpg" alt="Cachorro golden retriever correndo na praia">. Ele é importante para acessibilidade porque o leitor de tela lê esse texto em voz alta para usuários com deficiência visual, que de outra forma não teriam acesso à informação transmitida pela imagem.**

5. Os quatro princípios POUR:
P — Perceptível: o conteúdo deve ser apresentado de forma que todos os usuários possam percebê-lo, seja por visão, audição ou tato (ex.: legendas em vídeos).
O — Operável: todos os elementos interativos devem funcionar por diferentes meios de entrada, como teclado, não apenas mouse (ex.: navegar por Tab).
U — Compreensível: as informações e a interface devem ser fáceis de entender, com linguagem clara e comportamentos previsíveis (ex.: mensagens de erro explicativas).
R — Robusto: o conteúdo deve ser compatível com diferentes navegadores, dispositivos e tecnologias assistivas, inclusive futuras (ex.: HTML semântico correto).

3 Exemplos de Testes

Teste 1

Cenário: Usuário com deficiência visual
Como Testar: Usar NVDA ou VoiceOver
Objetivo: Verificar leitura correta dos elementos

Teste 2

Cenário: Formulário com dados inválidos
Como Testar: Inserir informações erradas
Objetivo: Validar mensagens de erro claras

Teste 3

Cenário: Navegação sem mouse
Como Testar: Usar apenas teclado
Objetivo: Garantir acessibilidade conforme WCAG

Fundamentos de Segurança da Informação nos Testes

A. Porque e Como?

Porquê:

Essencial para verificar a segurança das informações, Esse tipo de conhecimento nós permite proteger brechas comumente miradas por invasores ou até mesmo sem querer por usuários.

Como:

Se basear em testes criativos, se colocando no lugar do invasor, como um pensamento "preciso invadir esse sistema, e agora?" a partir disso com uma leve intuição e conhecimento de brechas você tenta propositalmente quebrar, invadir ou escapar de algo.
Até mesmo para iniciantes há listas na internet que dão conhecimento de brechas comumente utilizadas como OWASP top 10 ou testes de SQL INJECTION e Cross-site.

B. Uma visualização gráfica do tópico

C. 5 perguntas desafiadoras sobre o tópico

1. O que é o **OWASP Top 10** e qual sua importância no desenvolvimento de aplicações web?

2. Explique o que é uma vulnerabilidade de **SQL Injection** e como ela pode comprometer os dados de um sistema.

3. Qual a diferença entre **SAST**, **DAST** e **Pentest manual**? Quando usar cada um?

4. Por que a validação de entrada de dados é essencial para evitar falhas como **Cross-Site Scripting**? Dê um exemplo prático de como validar entradas.

5. Explique por que o **OWASP Top 10** é considerado um guia essencial para garantir a segurança e a performance de aplicações web.

Gabarito das Perguntas

1. OWASP Top 10

- O OWASP Top 10 é uma lista das vulnerabilidades mais críticas em aplicações web. Ele serve como guia para desenvolvedores e QAs identificarem e corrigirem riscos comuns, aumentando a segurança dos sistemas.

2. SQL Injection

- SQL Injection é uma falha onde o atacante insere comandos SQL maliciosos em entradas do sistema. Isso pode permitir acesso indevido ao banco de dados, vazamento ou alteração de informações.

3. SAST vs DAST vs Pentest

- SAST: analisa o código sem executar (fase de desenvolvimento).
- DAST: testa o sistema em execução (simula ataques).
- Pentest manual: feito por especialistas, identifica falhas mais complexas.

Cada um é usado em momentos diferentes para cobrir mais riscos.

4. Validação de Entrada / XSS

- A validação impede que dados maliciosos sejam processados. Sem isso, ataques como Cross-Site Scripting podem executar scripts no navegador do usuário.

Exemplo: validar tamanho, tipo e remover scripts de campos de texto.

5. Importância do OWASP Top 10

- O OWASP Top 10 é essencial porque foca nas ameaças mais reais e frequentes. Ele ajuda equipes a priorizar correções, proteger dados dos usuários e garantir sistemas mais seguros e confiáveis, impactando também na performance ao evitar falhas críticas.

D. 3 exemplos de teste contendo, Cenário, Como Testar e Objetivo

Teste 1: SQL INJECTION

Cenário: Um campo de login que exige usuário e senha

Como testar: Escreva em inputs normalmente que se posicionariam em comparações como Usuario e senha em interações com o banco, inserindo: " **'OR 1=1 --**"

Objetivo: Evitar SQL Injection, caso esse sistema permitisse um belo exemplo seria que todos os usuários seriam expostos

Teste 2: TESTE DE FORÇA BRUTA

Cenário: Campo de login

Como testar: Pode ser feito tanto com bots quanto com usuários ou scripts, chutando inúmeras senhas e usuários em um campo

Objetivo: O intuito é testar em como o sistema reage com inúmeras tentativas de acessar uma conta, testando proteção contra bots, requisitos de abertura funcionando e delay entre tentativas

Teste 3: PENTEST MANUAL

Cenário: Cadastro de produto, Nome do produto, Descrição, Preço
Essas informações são exibidas depois em uma página pública.

Como testar:
O pentester tenta identificar falhas na manipulação dos dados: Inserir scripts no campo descrição, Informar valores inválidos, Testar envio de dados muito grandes, Verificar se o sistema filtra corretamente entradas maliciosas, Observar como os dados são exibidos

Objetivo: Garantir que os dados inseridos são seguros, Evitar execução de código malicioso no navegador, Validar integridade das informações, Prevenir falhas que possam afetar usuários ou o sistema

Perguntas do Desafio Final

1. O que representam as métricas de performance "Throughput" e "Latência", e como elas são medidas utilizando ferramentas como JMeter, Gatling ou K6?

2. Quais são as principais diferenças, vantagens e desvantagens entre realizar um teste de usabilidade com usuários reais em comparação com um teste heurístico?

3. Qual é a diferença conceitual e prática entre as abordagens de avaliação de segurança SAST, DAST e o Pentest (teste de intrusão) manual?

Respostas do Desafio Final

1. *R: Throughput: quantidade de requisições processadas por unidade de tempo (ex: req/s).*

→ Medido no Apache JMeter, Gatling e k6 como taxa de requisições.
Latência: tempo que uma requisição leva para receber resposta (ex: ms).
→ Medido nas mesmas ferramentas como response time ou duração da requisição.

2. R: O teste heurístico é feito por especialistas que analisam a interface com base em regras como as Heurísticas de Nielsen. É rápido e barato, mas não reflete o comportamento real dos usuários.

O teste com usuários reais observa pessoas comuns executando tarefas no sistema. É mais caro e demorado, porém revela problemas genuínos que só aparecem no uso real.

A principal diferença é quem avalia: o especialista ou o próprio usuário. O ideal é combinar os dois — heurístico primeiro para corrigir problemas óbvios, teste com usuários depois para validar a experiência completa.

3.R:
O "SAST" é a verificação do sistema através apenas da observação do codigo sem sua execução.
(Basicamente um teste de White-Box)

O "DAST" é a verificação de uma parte do codigo a partir de sua execução e teste através da parte de fora do sistema(Front-end).
(Basicamente um teste de Black-Box)

O "PENTEST MANUAL" e feito através de todo complexo de um sistema, visando o seu todo, basicamente um "tenta invaidir ai meu mano"
(Utilizando tanto White, Black e Gray BOX)